



*Advanced AI-based dynamic Integration
of heterogeneous data for semantic
clusters formalisation in human-centric
intelligent enterprise systems*

Ontology-based supervised Transformers-like models for automatic triple extraction

Mario Lezoche

Thomas Pallet

Wednesday 18th February 2026

Table of content

01

**Introduction &
Pipeline overview**

02

**Dataset
creation**

03

Model training

04

**Evaluation
metrics**

05

**Results &
Analysis**

06

Conclusion

Table of content

01

**Introduction &
Pipeline overview**

02

**Dataset
creation**

03

Model training

04

**Evaluation
metrics**

05

**Results &
Analysis**

06

Conclusion

Introduction

Goal : develop and evaluate an ontology-based, supervised BERT-like pipeline (NLP, ...) for automatic triple extraction from text.

Pipeline overview

Goal : develop and evaluate an ontology-based, supervised BERT-like pipeline (NLP, ...) for automatic triple extraction from text.

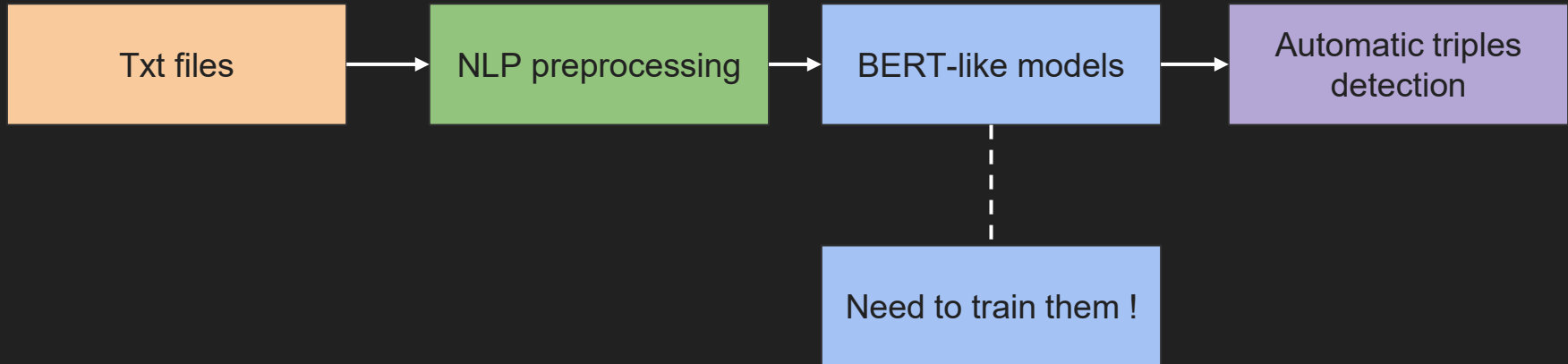


Table of content

01

**Introduction &
Pipeline overview**

02

**Dataset
creation**

03

Model training

04

**Evaluation
metrics**

05

**Results &
Analysis**

06

Conclusion

Dataset creation

- Use of LLMs to create stories, 300 stories (10 lines each)
- Last 50 with more complex knowledge, and always synonym for verbs that are meant to describe a relation (can learn, may learn, is able to learn, ...)
→ multiple templates per relation (would take time to do the per-rule based here)
- Stories are based on the ontology (classes, relations)

Dataset creation

- Entity mention detection
- Pair generation (important to do it bidirectional) because of the way relations works. Entity names E1->E2, only the correct direction is labelled. And added a “None” class for the other direction so that the model doesn't over classify triples.

- Entity marker injection :

```
{ "text": "[E1]Squirtle[/E1] is typed [E2]Water[/E2].", "label":  
  "http://example.org/pokemon#hasType" }
```

- JSONL exports then for training and validation

Dataset annotation

- Triples : (subject_URI, predicate_URI, object_URI), sentence
- Text are all annotated with JSON
- Feed the model with JSONL format for BERT (80% training and 20% in validation) which gives :
 - 8440 rows | 2188 rows
 - 'http://example.org/pokemon#hasMoveType': 650,
'http://example.org/pokemon#canLearnMove': 650,
'http://example.org/pokemon#ownsPokemon': 448,
'http://example.org/pokemon#hasType': 300,
'http://example.org/pokemon#evolvesTo': 181,
'http://example.org/pokemon#evolvesFrom': 181,
'http://example.org/pokemon#ownedBy': 152,

Table of content

01

**Introduction &
Pipeline overview**

02

**Dataset
creation**

03

Model training

04

**Evaluation
metrics**

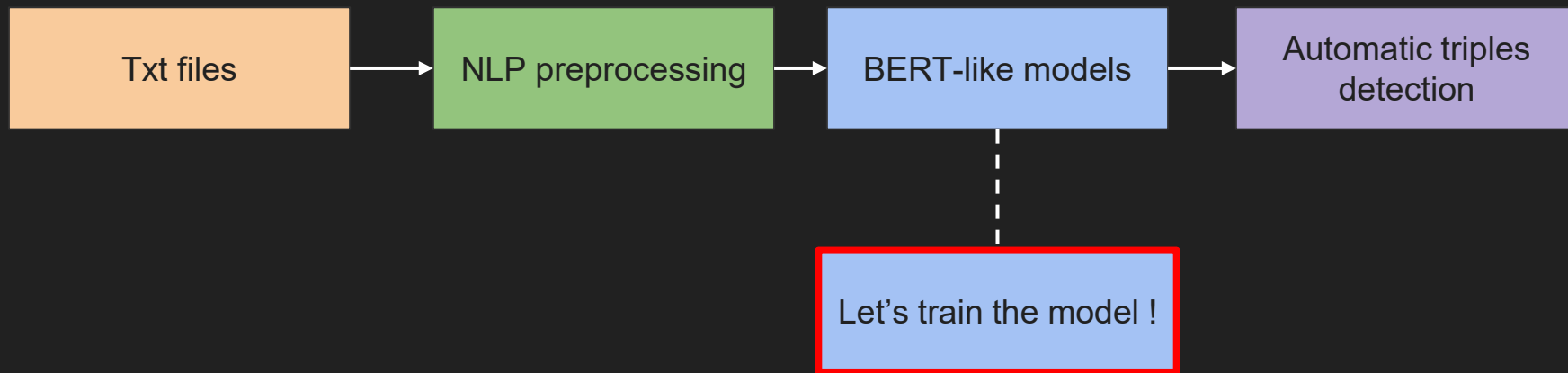
05

**Results &
Analysis**

06

Conclusion

Model training



⇒ Now that we have the dataset, let's train models and then inject them into the pipeline to evaluate them !

BERT: reminder

BERT's pre training:

- Encoder-only transformer
- Bidirectional attention
- Masked Language Modeling (MLM)
- Full attention layer across all tokens

ML/DL methods (fine-tuning):

- Can use scheduler for optimizer parameters (learning rates, ...)
- loss is the loss used for the pre trained models
- Each epoch, computed loss and accuracy for the training and the validation datasets.
- Weights for some classes if needed to prevent unbalancement in the result, micro f1 per class when trained

Model training

Use of pre trained models :

- XLNet: different pretraining paradigm (permutation autoregressive) , use more dependency tokens than BERT (autoregressive, use for better sequence modeling)
- BART: encoder-decoder denoising architecture so heavier than BERT
- DeROBERTa V3 (small): disentangled attention + improved decoder-style MLM pretraining (generally stronger token/context representation than BERT-base)
- T5 (small): text-to-text encoder-decoder transformer pretrained (usually heavier/slower than encoder-only classifiers for simple classification)
- BigBird: design for long documents like *Longformer*, attention mechanism different
- Funnel-transformer: hierarchical compression of tokens (encoder with progressive sequence length reduction), instead of constant token length all layers

→ Coverage of different transformers family

Why those models ?

Model	Pretraining Objective	Architecture	Attention Pattern	Efficiency Strategy
BERT	Masked language modeling (MLM)	Encoder-only	Dense full attention	No dedicated compression
XLNet	Permutation language modeling	Encoder-style (classification)	Dense full attention	No dedicated compression
BART	Denoising autoencoding	Encoder-decoder	Dense full attention	No dedicated compression
BigBird	MLM-style variant	Encoder-style (classification)	Sparse local + random + global	Sparse attention for long context
Funnel Transformer	MLM-style variant	Encoder-style (classification)	Dense full attention	Compressed hierarchy (sequence length reduction)

Model example: XLNet

```

XLNetForSequenceClassification(
  (transformer): XLNetModel(
    (word_embedding): Embedding(32000, 768)
    (layer): ModuleList(
      (0-11): 12 x XLNetLayer(
        (rel_attn): XLNetRelativeAttention(
          (layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (dropout): Dropout(p=0.1, inplace=False)
        )
        (ff): XLNetFeedForward(
          (layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (layer_1): Linear(in_features=768, out_features=3072, bias=True)
          (layer_2): Linear(in_features=3072, out_features=768, bias=True)
          (dropout): Dropout(p=0.1, inplace=False)
          (activation_function): GELUActivation()
        )
        (dropout): Dropout(p=0.1, inplace=False)
      )
    )
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (sequence_summary): XLNetSequenceSummary(
    (summary): Linear(in_features=768, out_features=768, bias=True)
    (activation): Tanh()
    (first_dropout): Identity()
    (last_dropout): Dropout(p=0.1, inplace=False)
  )
  (logits_proj): Linear(in_features=768, out_features=8, bias=True)
)

```

Model example: BART

```

BartForSequenceClassification(
  (model): BartModel(
    (shared): BartScaledWordEmbedding(50265, 768, padding_idx=1)
    (encoder): BartEncoder(
      (embed_tokens): BartScaledWordEmbedding(50265, 768, padding_idx=1)
      (embed_positions): BartLearnedPositionalEmbedding(1026, 768)
      (layers): ModuleList(
        (0-5): 6 x BartEncoderLayer(
          (self_attn): BartAttention(
            (k_proj): Linear(in_features=768, out_features=768, bias=True)
            (v_proj): Linear(in_features=768, out_features=768, bias=True)
            (q_proj): Linear(in_features=768, out_features=768, bias=True)
            (out_proj): Linear(in_features=768, out_features=768, bias=True)
          )
          (self_attn_layer_norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
          (activation_fn): GELUActivation()
          (fc1): Linear(in_features=768, out_features=3072, bias=True)
          (fc2): Linear(in_features=3072, out_features=768, bias=True)
          (final_layer_norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        )
      )
      (layernorm_embedding): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
    )
    (decoder): BartDecoder(
      (embed_tokens): BartScaledWordEmbedding(50265, 768, padding_idx=1)
      (embed_positions): BartLearnedPositionalEmbedding(1026, 768)
      (layers): ModuleList(
        (0-5): 6 x BartDecoderLayer(
          (self_attn): BartAttention(
            (k_proj): Linear(in_features=768, out_features=768, bias=True)
            (v_proj): Linear(in_features=768, out_features=768, bias=True)
            (q_proj): Linear(in_features=768, out_features=768, bias=True)
            (out_proj): Linear(in_features=768, out_features=768, bias=True)
          )
          (activation_fn): GELUActivation()
          (self_attn_layer_norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
          (encoder_attn): BartAttention(
            (k_proj): Linear(in_features=768, out_features=768, bias=True)
            (v_proj): Linear(in_features=768, out_features=768, bias=True)
            (q_proj): Linear(in_features=768, out_features=768, bias=True)
            (out_proj): Linear(in_features=768, out_features=768, bias=True)
          )
          (encoder_attn_layer_norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
          (fc1): Linear(in_features=768, out_features=3072, bias=True)
          (fc2): Linear(in_features=3072, out_features=768, bias=True)
          (final_layer_norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        )
      )
      (layernorm_embedding): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
    )
  )
  (classification_head): BartClassificationHead(
    (dense): Linear(in_features=768, out_features=768, bias=True)
    (dropout): Dropout(p=0.0, inplace=False)
    (out_proj): Linear(in_features=768, out_features=8, bias=True)
  )
)

```

Model example: DeROBERTa

```

DebertaV2ForSequenceClassification(
  (deberta): DebertaV2Model(
    (embeddings): DebertaV2Embeddings(
      (word_embeddings): Embedding(128100, 768, padding_idx=0)
      (LayerNorm): LayerNorm((768,), eps=1e-07, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): DebertaV2Encoder(
      (layer): ModuleList(
        (0-5): 6 x DebertaV2Layer(
          (attention): DebertaV2Attention(
            (self): DisentangledSelfAttention(
              (query_proj): Linear(in_features=768, out_features=768, bias=True)
              (key_proj): Linear(in_features=768, out_features=768, bias=True)
              (value_proj): Linear(in_features=768, out_features=768, bias=True)
              (pos_dropout): Dropout(p=0.1, inplace=False)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): DebertaV2SelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-07, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): DebertaV2Intermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): DebertaV2Output(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-07, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
      (rel_embeddings): Embedding(512, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-07, elementwise_affine=True)
    )
    (pooler): ContextPooler(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (dropout): Dropout(p=0, inplace=False)
    )
    (classifier): Linear(in_features=768, out_features=8, bias=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
)

```

Model example: T5

```
T5ForSequenceClassification(
  (transformer): T5Model(
    (shared): Embedding(32128, 512)
    (encoder): T5Stack(
      (embed_tokens): Embedding(32128, 512)
      (block): ModuleList(
        (0): T5Block(
          (layer): ModuleList(
            (0): T5LayerSelfAttention(
              (SelfAttention): T5Attention(
                (q): Linear(in_features=512, out_features=512, bias=False)
                (k): Linear(in_features=512, out_features=512, bias=False)
                (v): Linear(in_features=512, out_features=512, bias=False)
                (o): Linear(in_features=512, out_features=512, bias=False)
                (relative_attention_bias): Embedding(32, 8)
              )
              (layer_norm): T5LayerNorm()
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (1): T5LayerFF(
              (DenseReluDense): T5DenseActDense(
                (wi): Linear(in_features=512, out_features=2048, bias=False)
                (wo): Linear(in_features=2048, out_features=512, bias=False)
                (dropout): Dropout(p=0.1, inplace=False)
                (act): ReLU()
              )
              (layer_norm): T5LayerNorm()
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
        )
      )
    )
  )
  (1-5): 5 x T5Block(
    (layer): ModuleList(
      (0): T5LayerSelfAttention(
        (SelfAttention): T5Attention(
          (q): Linear(in_features=512, out_features=512, bias=False)
          (k): Linear(in_features=512, out_features=512, bias=False)
          (v): Linear(in_features=512, out_features=512, bias=False)
          (o): Linear(in_features=512, out_features=512, bias=False)
```

■ ■ ■

```
    (final_layer_norm): T5LayerNorm()
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (classification_head): T5ClassificationHead(
    (dense): Linear(in_features=512, out_features=512, bias=True)
    (dropout): Dropout(p=0.0, inplace=False)
    (out_proj): Linear(in_features=512, out_features=8, bias=True)
  )
)
```

Model example: BigBird

```
BigBirdForSequenceClassification(
  (bert): BigBirdModel(
    (embeddings): BigBirdEmbeddings(
      (word_embeddings): Embedding(50358, 768, padding_idx=0)
      (position_embeddings): Embedding(4096, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BigBirdEncoder(
      (layer): ModuleList(
        (0-11): 12 x BigBirdLayer(
          (attention): BigBirdAttention(
            (self): BigBirdBlockSparseAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
            )
            (output): BigBirdSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): BigBirdIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): NewGELUActivation()
          )
          (output): BigBirdOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
      (pooler): Linear(in_features=768, out_features=768, bias=True)
      (activation): Tanh()
    )
    (classifier): BigBirdClassificationHead(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (dropout): Dropout(p=0.1, inplace=False)
      (out_proj): Linear(in_features=768, out_features=8, bias=True)
    )
  )
)
```

Model example: Funnel Transformer

```

FunnelForSequenceClassification(
  (funnel): FunnelBaseModel(
    (embeddings): FunnelEmbeddings(
      (word_embeddings): Embedding(30522, 768)
      (layer_norm): LayerNorm((768,)), eps=1e-09, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): FunnelEncoder(
      (attention_structure): FunnelAttentionStructure(
        (sin_dropout): Dropout(p=0.1, inplace=False)
        (cos_dropout): Dropout(p=0.1, inplace=False)
      )
      (blocks): ModuleList(
        (0-2): 3 x ModuleList(
          (0-3): 4 x FunnelLayer(
            (attention): FunnelRelMultiheadAttention(
              (hidden_dropout): Dropout(p=0.1, inplace=False)
              (attention_dropout): Dropout(p=0.1, inplace=False)
              (q_head): Linear(in_features=768, out_features=768, bias=False)
              (k_head): Linear(in_features=768, out_features=768, bias=True)
              (v_head): Linear(in_features=768, out_features=768, bias=True)
              (post_proj): Linear(in_features=768, out_features=768, bias=True)
              (layer_norm): LayerNorm((768,)), eps=1e-09, elementwise_affine=True)
            )
            (ffn): FunnelPositionwiseFFN(
              (linear_1): Linear(in_features=768, out_features=3072, bias=True)
              (activation_function): NewGELUActivation()
              (activation_dropout): Dropout(p=0.0, inplace=False)
              (linear_2): Linear(in_features=3072, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
              (layer_norm): LayerNorm((768,)), eps=1e-09, elementwise_affine=True)
            )
          )
        )
      )
    )
  )
  (classifier): FunnelClassificationHead(
    (linear_hidden): Linear(in_features=768, out_features=768, bias=True)
    (dropout): Dropout(p=0.1, inplace=False)
    (linear_out): Linear(in_features=768, out_features=8, bias=True)
  )
)

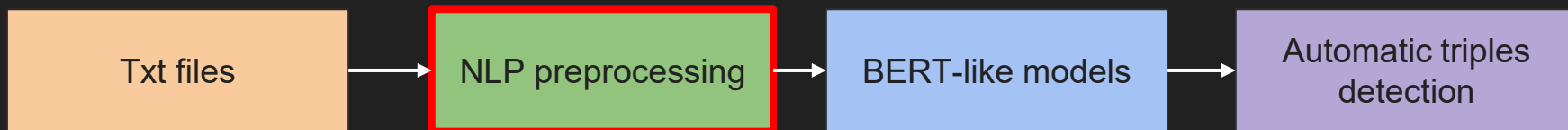
```

Recap of the pipeline



- 10 stories of 30 lines each from the previous work (benchmark unsupervised)
- Annotations in JSON, we have outputs triples for the evaluation at the end

Recap of the pipeline



- Entity recognition (dictionary based on the OWL = ontology)
- Named entities E1 or E2 + domains and range for multiple E1 and E2
- Sentence includes now the mention of E1 and E2 for the BERT-models

Bulbasaur is a Grass and Poison type, at least in his notes.

Bulbasaur **POKEMON_SPECIES** is a **Grass TYPE** and **Poison TYPE** type, at least in his notes.

ACTUAL TRIPLES:

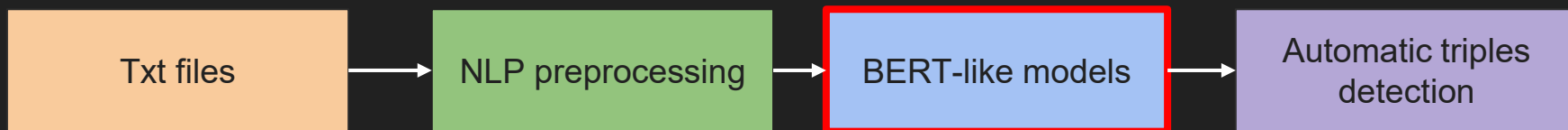
```
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Grass')
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Poison')
```

PREDICTED TRIPLES:

```
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Grass')
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Poison')
```

OK

Recap of the pipeline



- Input :
Text annotated with entity E1/E2, vectorization of the whole sentence
- Output :
Classification problem so it will give us a class output

Then, we know both entities, the orders between them (E1 and E2) so we can create the triples E1, relation, E2 !

Recap of the pipeline



- BERT-like model gave us a class as the output

Then, we know both entities, the orders between them (E1 and E2) so we can create the triples E1, relation, E2 ! (if output class is not the None class)

Table of content

01

**Introduction &
Pipeline overview**

02

**Dataset
creation**

03

Model training

04

**Evaluation
metrics**

05

**Results &
Analysis**

06

Conclusion

Evaluation metrics

In this case, relation is a classification problem at the triple level.

We compare then the actual vs predicted triples from each sentence from each story

- **TP (True Positive)**: predicted triple exists in actual
- **FP (False Positive)**: predicted triple not in actual
- **FN (False Negative)**: actual triple not predicted
- **Precision** = $TP / (TP + FP)$

“How many predicted relations are correct?”

- **Recall** = $TP / (TP + FN)$

“How many actual relations did we recover?”

- **F1** = $2PR / (P + R) \rightarrow$ important

Balance between precision and recall

Table of content

01

**Introduction &
Pipeline overview**

02

**Dataset
creation**

03

Model training

04

**Evaluation
metrics**

05

**Results &
Analysis**

06

Conclusion

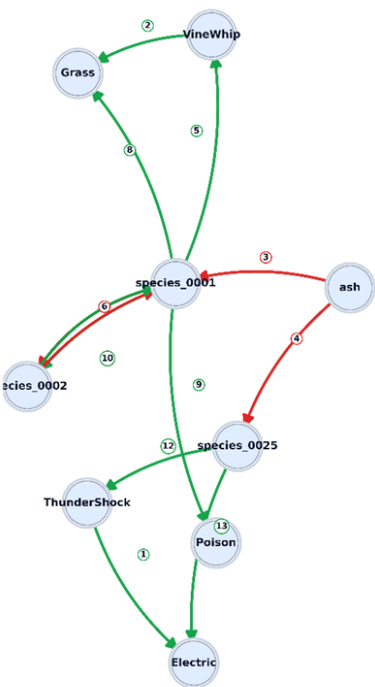
Results XLNet model

Story	TP	FP	FN	Precision (%)	Recall (%)	F1 (%)
story_01	9	4	0	69.2 %	100 %	81.8 %
story_02	10	6	0	62.5 %	100 %	76.9 %
story_03	11	3	0	78.6 %	100 %	88.0 %
story_04	11	4	3	73.3 %	78.6 %	75.9 %
story_05	12	3	5	80.0 %	70.6 %	75.0 %
story_06	16	2	1	88.9 %	94.1 %	91.4 %
story_07	12	4	6	75.0 %	66.7 %	70.6 %
story_08	12	2	3	85.7 %	80.0 %	82.8 %
story_09	14	3	3	82.4 %	82.4 %	82.4 %
story_10	18	4	5	81.8 %	78.3 %	80.0 %
TOTAL (mean)	-	-	-	78.1 %	82.8 %	80.4 %

Results XLNet model: story 1

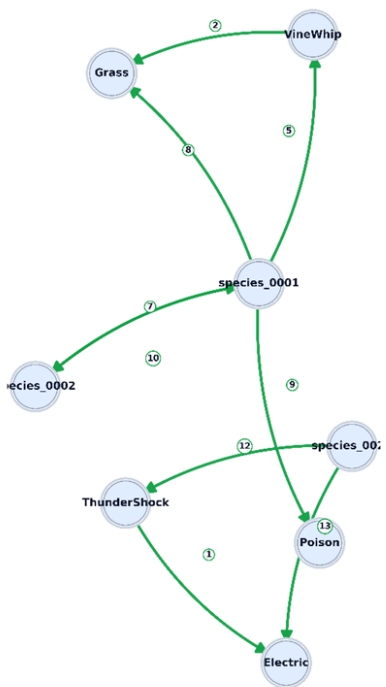
Relation Graph Comparison - story_01 | Model=XLNET-BALANCED | TP=9 FP=4 FN=0 | P=0.692 R=1.000 F1=0.818

Predicted Graph - story_01 (green=TP, red=FP)



- Edge list**
- 01. ThunderShock -> Electric : hasMoveType [TP]
 - 02. VineWhip -> Grass : hasMoveType [TP]
 - 03. ash -> species_0001 : ownsPokemon [FP]
 - 04. ash -> species_0025 : ownsPokemon [FP]
 - 05. species_0001 -> VineWhip : canLearnMove [TP]
 - 06. species_0001 -> species_0002 : evolvesFrom [FP]
 - 07. species_0001 -> species_0002 : evolvesTo [TP]
 - 08. species_0001 -> Grass : hasType [TP]
 - 09. species_0001 -> Poison : hasType [TP]
 - 10. species_0002 -> species_0001 : evolvesFrom [TP]
 - 11. species_0002 -> species_0001 : evolvesTo [FP]
 - 12. species_0025 -> ThunderShock : canLearnMove [TP]
 - 13. species_0025 -> Electric : hasType [TP]

Actual Graph - story_01 (green=TP, orange=FN)

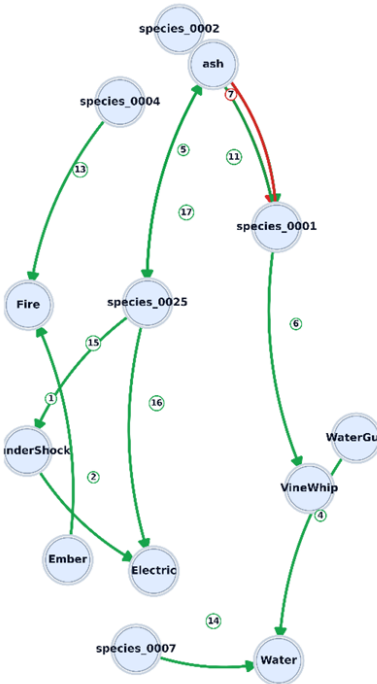


- Edge list**
- 01. ThunderShock -> Electric : hasMoveType [TP]
 - 02. VineWhip -> Grass : hasMoveType [TP]
 - 05. species_0001 -> VineWhip : canLearnMove [TP]
 - 07. species_0001 -> species_0002 : evolvesTo [TP]
 - 08. species_0001 -> Grass : hasType [TP]
 - 09. species_0001 -> Poison : hasType [TP]
 - 10. species_0002 -> species_0001 : evolvesFrom [TP]
 - 12. species_0025 -> ThunderShock : canLearnMove [TP]
 - 13. species_0025 -> Electric : hasType [TP]

Results XLNet model: story 8

Relation Graph Comparison - story_08 | Model=XLNET-BALANCED | TP=12 FP=2 FN=3 | P=0.857 R=0.800 F1=0.828

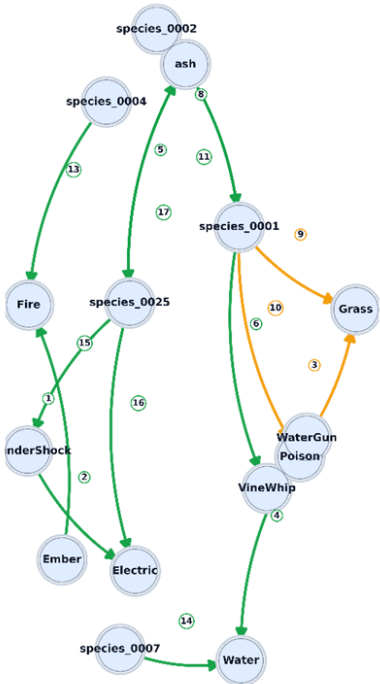
Predicted Graph - story_08 (green=TP, red=FP)



Edge list

- 01. Ember -> Fire : hasMoveType [TP]
- 02. ThunderShock -> Electric : hasMoveType [TP]
- 04. WaterGun -> Water : hasMoveType [TP]
- 05. ash -> species_0025 : ownsPokemon [TP]
- 06. species_0001 -> VineWhip : canLearnMove [TP]
- 07. species_0001 -> species_0002 : evolvesFrom [FP]
- 08. species_0001 -> species_0002 : evolvesTo [TP]
- 11. species_0002 -> species_0001 : evolvesFrom [TP]
- 12. species_0002 -> species_0001 : evolvesTo [FP]
- 13. species_0004 -> Fire : hasType [TP]
- 14. species_0007 -> Water : hasType [TP]
- 15. species_0025 -> ThunderShock : canLearnMove [TP]
- 16. species_0025 -> Electric : hasType [TP]
- 17. species_0025 -> ash : ownedBy [TP]

Actual Graph - story_08 (green=TP, orange=FN)



Edge list

- 01. Ember -> Fire : hasMoveType [TP]
- 02. ThunderShock -> Electric : hasMoveType [TP]
- 03. VineWhip -> Grass : hasMoveType [FN]
- 04. WaterGun -> Water : hasMoveType [TP]
- 05. ash -> species_0025 : ownsPokemon [TP]
- 06. species_0001 -> VineWhip : canLearnMove [TP]
- 08. species_0001 -> species_0002 : evolvesTo [TP]
- 09. species_0001 -> Grass : hasType [FN]
- 10. species_0001 -> Poison : hasType [FN]
- 11. species_0002 -> species_0001 : evolvesFrom [TP]
- 13. species_0004 -> Fire : hasType [TP]
- 14. species_0007 -> Water : hasType [TP]
- 15. species_0025 -> ThunderShock : canLearnMove [TP]
- 16. species_0025 -> Electric : hasType [TP]
- 17. species_0025 -> ash : ownedBy [TP]

Results XLNet model

→ Problem with FP, Ash always detected as an entity and model always tries to link it to another entity even tho it has to be a 'NONE' relations because they are no link into my text

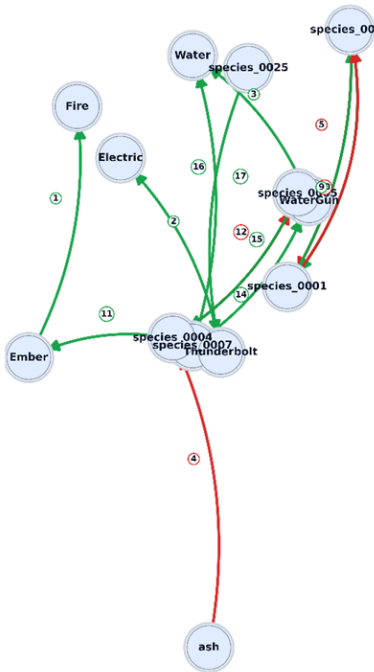
Results BART model

Story	TP	FP	FN	Precision (%)	Recall (%)	F1 (%)
story_01	9	4	0	69.2 %	100 %	81.8 %
story_02	10	5	0	66.7 %	100 %	80.0 %
story_03	10	3	1	76.9 %	90.9 %	83.3 %
story_04	10	4	4	71.4 %	71.4 %	71.4 %
story_05	12	3	5	80.0 %	70.6 %	75.0 %
story_06	16	2	1	88.9 %	94.1 %	91.4 %
story_07	15	4	3	78.9 %	83.3 %	81.1 %
story_08	13	2	2	86.7 %	86.7 %	86.7 %
story_09	16	4	1	80.0 %	94.1 %	86.5 %
story_10	22	4	1	84.6 %	95.7 %	89.8 %
TOTAL (mean)	-	-	-	79.2 %	88.1 %	83.4 %

Results BART model: story 4

Relation Graph Comparison - story_04 | Model=BART | TP=10 FP=4 FN=4 | P=0.714 R=0.714 F1=0.714

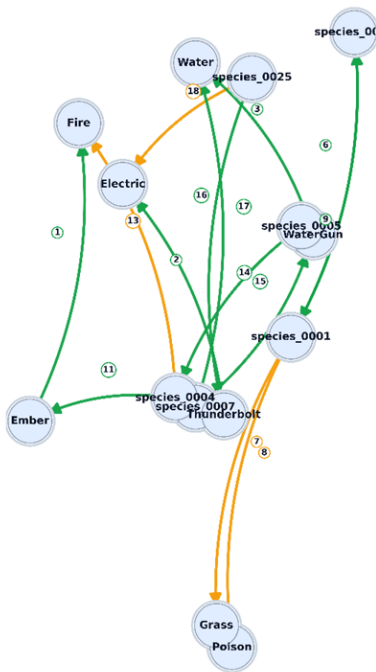
Predicted Graph - story_04 (green=TP, red=FP)



Edge list

- 01. Ember -> Fire : hasMoveType [TP]
- 02. Thunderbolt -> Electric : hasMoveType [TP]
- 03. WaterGun -> Water : hasMoveType [TP]
- 04. ash -> species_0004 : ownsPokemon [FP]
- 05. species_0001 -> species_0002 : evolvesFrom [FP]
- 06. species_0001 -> species_0002 : evolvesTo [TP]
- 09. species_0002 -> species_0001 : evolvesFrom [TP]
- 10. species_0002 -> species_0001 : evolvesTo [FP]
- 11. species_0004 -> Ember : canLearnMove [TP]
- 12. species_0004 -> species_0005 : evolvesFrom [FP]
- 14. species_0005 -> species_0004 : evolvesFrom [TP]
- 15. species_0007 -> WaterGun : canLearnMove [TP]
- 16. species_0007 -> Water : hasType [TP]
- 17. species_0025 -> Thunderbolt : canLearnMove [TP]

Actual Graph - story_04 (green=TP, orange=FN)



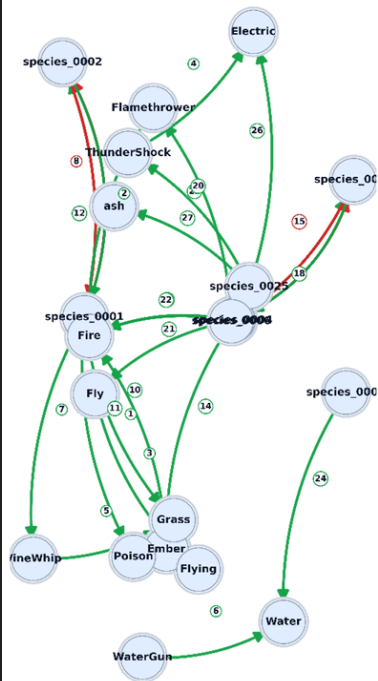
Edge list

- 01. Ember -> Fire : hasMoveType [TP]
- 02. Thunderbolt -> Electric : hasMoveType [TP]
- 03. WaterGun -> Water : hasMoveType [TP]
- 06. species_0001 -> species_0002 : evolvesTo [TP]
- 07. species_0001 -> Grass : hasType [FN]
- 08. species_0001 -> Poison : hasType [FN]
- 09. species_0002 -> species_0001 : evolvesFrom [TP]
- 11. species_0004 -> Ember : canLearnMove [TP]
- 13. species_0004 -> Fire : hasType [FN]
- 14. species_0005 -> species_0004 : evolvesFrom [TP]
- 15. species_0007 -> WaterGun : canLearnMove [TP]
- 16. species_0007 -> Water : hasType [TP]
- 17. species_0025 -> Thunderbolt : canLearnMove [TP]
- 18. species_0025 -> Electric : hasType [FN]

Results BART model: story 10

Relation Graph Comparison - story_10 | Model=BART | TP=22 FP=4 FN=1 | P=0.846 R=0.957 F1=0.898

Predicted Graph - story_10 (green=TP, red=FP)

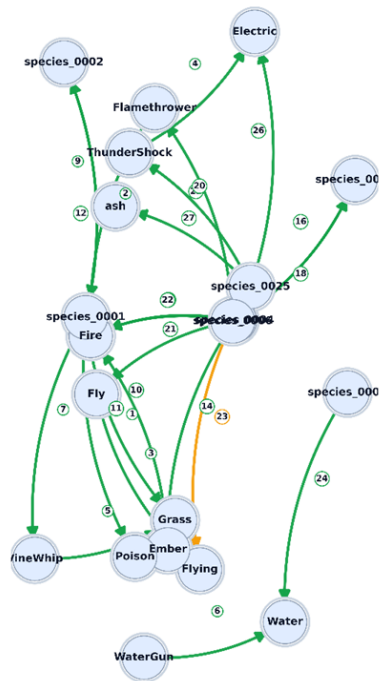


Edge list

- 01. Ember -> Fire : hasMoveType [TP]
- 02. Flamethrower -> Fire : hasMoveType [TP]
- 03. Fly -> Flying : hasMoveType [TP]
- 04. ThunderShock -> Electric : hasMoveType [TP]
- 05. VineWhip -> Grass : hasMoveType [TP]
- 06. WaterGun -> Water : hasMoveType [TP]
- 07. species_0001 -> VineWhip : canLearnMove [TP]
- 08. species_0001 -> species_0002 : evolvesFrom [FP]
- 09. species_0001 -> species_0002 : evolvesTo [TP]
- 10. species_0001 -> Grass : hasType [TP]
- 11. species_0001 -> Poison : hasType [TP]
- 12. species_0002 -> species_0001 : evolvesFrom [TP]
- 13. species_0002 -> species_0001 : evolvesTo [FP]
- 14. species_0004 -> Ember : canLearnMove [TP]
- 15. species_0004 -> species_0005 : evolvesFrom [FP]
- 16. species_0004 -> species_0005 : evolvesTo [TP]
- 17. species_0004 -> Fire : hasType [TP]
- 18. species_0005 -> species_0004 : evolvesFrom [TP]
- 19. species_0005 -> species_0004 : evolvesTo [FP]
- 20. species_0006 -> Flamethrower : canLearnMove [TP]

...

Actual Graph - story_10 (green=TP, orange=FN)



Edge list

- 01. Ember -> Fire : hasMoveType [TP]
- 02. Flamethrower -> Fire : hasMoveType [TP]
- 03. Fly -> Flying : hasMoveType [TP]
- 04. ThunderShock -> Electric : hasMoveType [TP]
- 05. VineWhip -> Grass : hasMoveType [TP]
- 06. WaterGun -> Water : hasMoveType [TP]
- 07. species_0001 -> VineWhip : canLearnMove [TP]
- 09. species_0001 -> species_0002 : evolvesTo [TP]
- 10. species_0001 -> Grass : hasType [TP]
- 11. species_0001 -> Poison : hasType [TP]
- 12. species_0002 -> species_0001 : evolvesFrom [TP]
- 14. species_0004 -> Ember : canLearnMove [TP]
- 16. species_0004 -> species_0005 : evolvesTo [TP]
- 17. species_0004 -> Fire : hasType [TP]
- 18. species_0005 -> species_0004 : evolvesFrom [TP]
- 20. species_0006 -> Flamethrower : canLearnMove [TP]
- 21. species_0006 -> Fly : canLearnMove [TP]
- 22. species_0006 -> Fire : hasType [TP]
- 23. species_0006 -> Flying : hasType [FN]
- 24. species_0007 -> Water : hasType [TP]

...

TP (correct prediction) FP (added / wrong prediction) FN (missing expected relation)

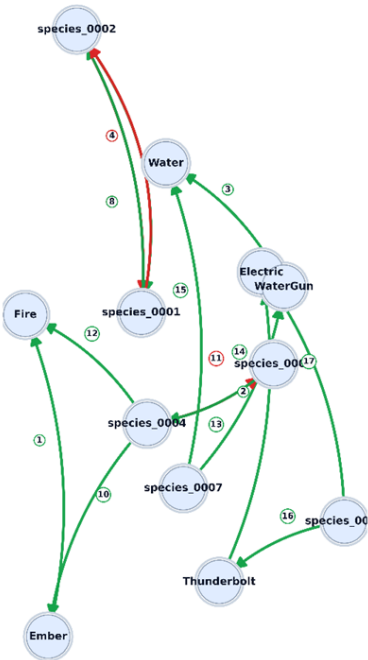
Results DeROBERTa (small) model

Story	TP	FP	FN	Precision (%)	Recall (%)	F1 (%)
story_01	9	4	0	69.2 %	100 %	81.8 %
story_02	10	6	0	62.5 %	100 %	76.9 %
story_03	11	3	0	78.6 %	100 %	88.0 %
story_04	12	3	2	80.0 %	85.7 %	82.8 %
story_05	15	3	2	83.3 %	88.2 %	85.7 %
story_06	17	2	0	89.5 %	100 %	94.4 %
story_07	17	6	1	73.9 %	94.4 %	82.9 %
story_08	15	2	0	88.2 %	100 %	93.8 %
story_09	16	4	1	80.0 %	94.1 %	86.5 %
story_10	22	6	1	78.6 %	95.7 %	86.3 %
TOTAL (mean)	-	-	-	78.7 %	95.4 %	86.2 %

Results DeROBERTa model: story 4

Relation Graph Comparison - story_04 | Model=DEROBERTA-V3-SMALL | TP=12 FP=3 FN=2 | P=0.800 R=0.857 F1=0.828

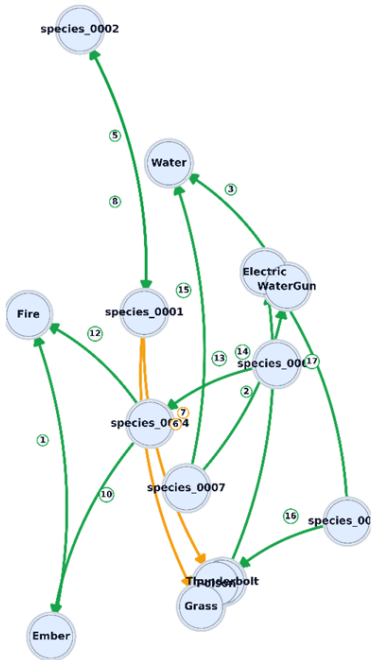
Predicted Graph - story_04 (green=TP, red=FP)



Edge list

- 01. Ember -> Fire : hasMoveType [TP]
- 02. Thunderbolt -> Electric : hasMoveType [TP]
- 03. WaterGun -> Water : hasMoveType [TP]
- 04. species_0001 -> species_0002 : evolvesFrom [FP]
- 05. species_0001 -> species_0002 : evolvesTo [TP]
- 08. species_0002 -> species_0001 : evolvesFrom [TP]
- 09. species_0002 -> species_0001 : evolvesTo [FP]
- 10. species_0004 -> Ember : canLearnMove [TP]
- 11. species_0004 -> species_0005 : evolvesFrom [FP]
- 12. species_0004 -> Fire : hasType [TP]
- 13. species_0005 -> species_0004 : evolvesFrom [TP]
- 14. species_0007 -> WaterGun : canLearnMove [TP]
- 15. species_0007 -> Water : hasType [TP]
- 16. species_0025 -> Thunderbolt : canLearnMove [TP]
- 17. species_0025 -> Electric : hasType [TP]

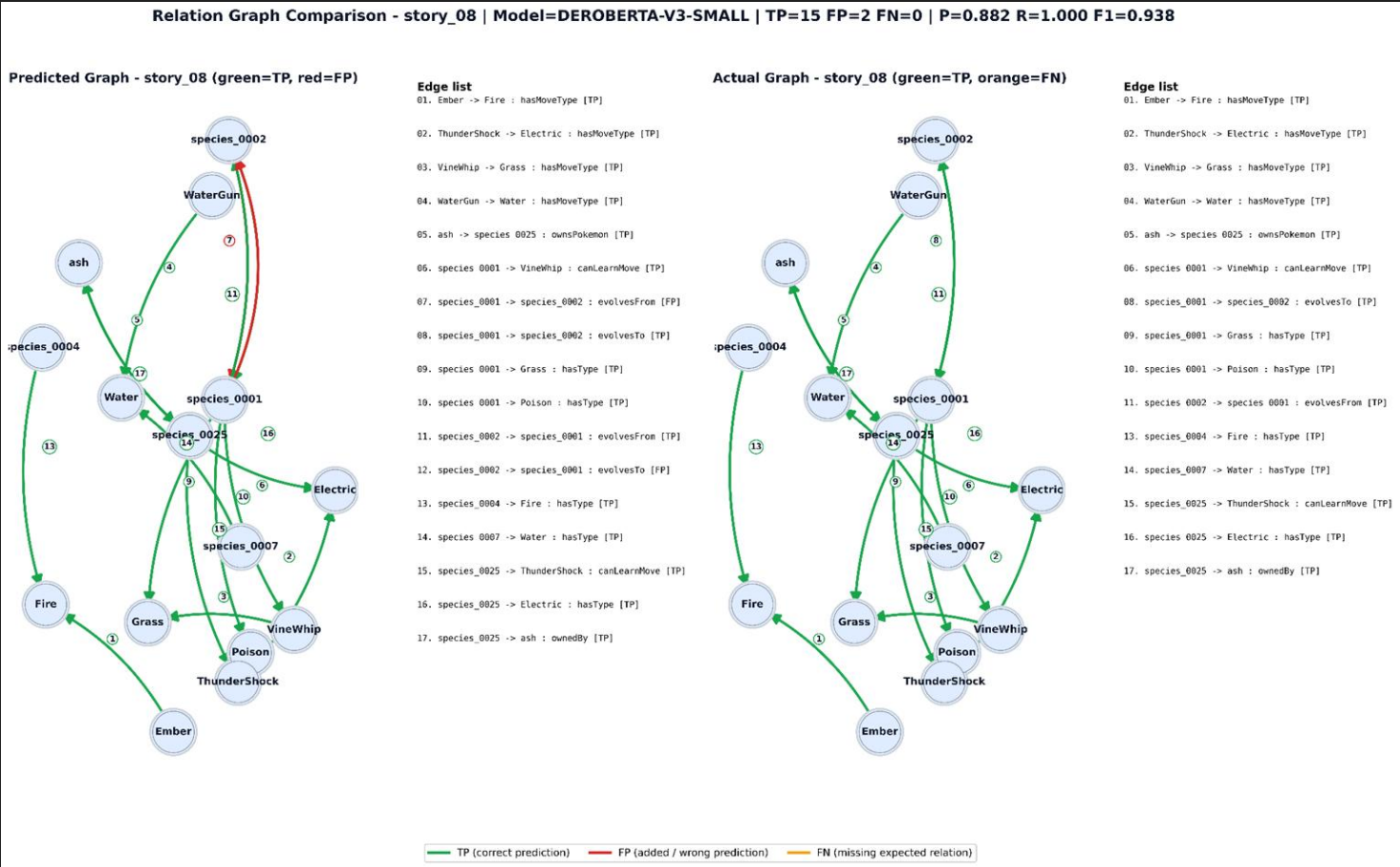
Actual Graph - story_04 (green=TP, orange=FN)



Edge list

- 01. Ember -> Fire : hasMoveType [TP]
- 02. Thunderbolt -> Electric : hasMoveType [TP]
- 03. WaterGun -> Water : hasMoveType [TP]
- 05. species_0001 -> species_0002 : evolvesTo [TP]
- 06. species_0001 -> Grass : hasType [FN]
- 07. species_0001 -> Poison : hasType [FN]
- 08. species_0002 -> species_0001 : evolvesFrom [TP]
- 10. species_0004 -> Ember : canLearnMove [TP]
- 12. species_0004 -> Fire : hasType [TP]
- 13. species_0005 -> species_0004 : evolvesFrom [TP]
- 14. species_0007 -> WaterGun : canLearnMove [TP]
- 15. species_0007 -> Water : hasType [TP]
- 16. species_0025 -> Thunderbolt : canLearnMove [TP]
- 17. species_0025 -> Electric : hasType [TP]

Results DeROBERTa model: story 8



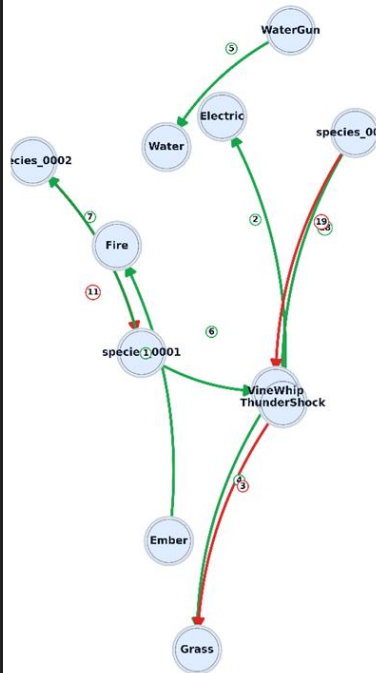
Results T5 (small) model

Story	TP	FP	FN	Precision (%)	Recall (%)	F1 (%)
story_01	6	3	3	66.7 %	66.7 %	66.7 %
story_02	8	3	2	72.7 %	80.0 %	76.2 %
story_03	6	1	5	85.7 %	54.5 %	66.7 %
story_04	6	3	8	66.7 %	42.9 %	52.2 %
story_05	9	2	8	81.8 %	52.9 %	64.3 %
story_06	7	1	10	87.5 %	41.2 %	56.0 %
story_07	7	3	11	70.0 %	38.9 %	50.0 %
story_08	7	1	8	87.5 %	46.7 %	60.9 %
story_09	8	2	9	80.0 %	47.1 %	59.3 %
story_10	11	2	12	84.6 %	47.6 %	61.1 %
TOTAL (mean)	-	-	-	78.1 %	49.7 %	60.7 %

Results T5 (small) model: story 7

Relation Graph Comparison - story_07 | Model=T5-SMALL | TP=7 FP=3 FN=11 | P=0.700 R=0.389 F1=0.500

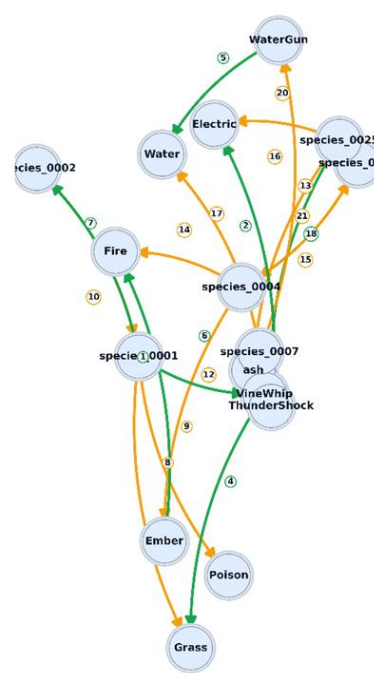
Predicted Graph - story_07 (green=TP, red=FP)



Edge list

```
01. Ember -> Fire : hasMoveType [TP]
02. ThunderShock -> Electric : hasMoveType [TP]
03. ThunderShock -> Grass : hasMoveType [FP]
04. VineWhip -> Grass : hasMoveType [TP]
05. WaterGun -> Water : hasMoveType [TP]
06. species_0001 -> VineWhip : canLearnMove [TP]
07. species_0001 -> species_0002 : evolvesTo [TP]
11. species_0002 -> species_0001 : evolvesTo [FP]
18. species_0025 -> ThunderShock : canLearnMove [TP]
19. species_0025 -> VineWhip : canLearnMove [FP]
```

Actual Graph - story_07 (green=TP, orange=FN)



Edge list

01. Ember -> Fire : hasMoveType [TP]
02. ThunderShock -> Electric : hasMoveType [TP]
04. VineWhip -> Grass : hasMoveType [TP]
05. WaterGun -> Water : hasMoveType [TP]
06. species_0001 -> VineWhip : canLearnMove [TP]
07. species_0001 -> species_0002 : evolvesTo [TP]
09. species_0001 -> Grass : hasType [FN]
09. species_0001 -> Poison : hasType [FN]
10. species_0002 -> species_0001 : evolvesFrom [FN]
12. species_0004 -> Ember : canLearnMove [FN]
13. species_0004 -> species_0005 : evolvesTo [FN]
14. species_0004 -> Fire : hasType [FN]
15. species_0005 -> species_0004 : evolvesFrom [FN]
16. species_0007 -> WaterGun : canLearnMove [FN]
17. species_0007 -> Water : hasType [FN]
18. species_0025 -> ThunderShock : canLearnMove [TP]
20. species_0025 -> Electric : hasType [FN]
21. species_0025 -> ash : ownedBy [FN]

— TP (correct prediction) — FP (added / wrong prediction) — FN (missing expected relation)

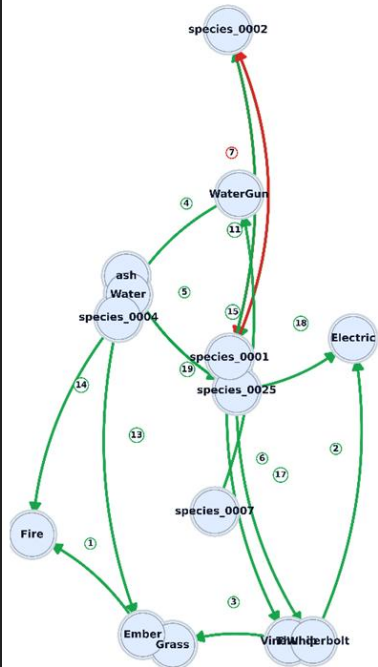
Results BigBird model

Story	TP	FP	FN	Precision (%)	Recall (%)	F1 (%)
story_01	9	4	0	69.2 %	100 %	81.8 %
story_02	10	5	0	66.7 %	100 %	80.0 %
story_03	10	3	1	76.9 %	90.9 %	83.3 %
story_04	12	3	2	80.0 %	85.7 %	82.8 %
story_05	13	3	4	81.2 %	76.5 %	78.8 %
story_06	15	2	2	88.2 %	88.2 %	88.2 %
story_07	13	4	5	76.5 %	72.2 %	74.3 %
story_08	11	2	4	84.6 %	73.3 %	78.6 %
story_09	14	2	3	87.5 %	82.4 %	84.8 %
story_10	19	4	4	82.6 %	82.6 %	82.6 %
TOTAL (mean)	-	-	-	79.7 %	83.4 %	81.6 %

Results BigBird model: story 9

Relation Graph Comparison - story_09 | Model=BIGBIRD | TP=14 FP=2 FN=3 | P=0.875 R=0.824 F1=0.848

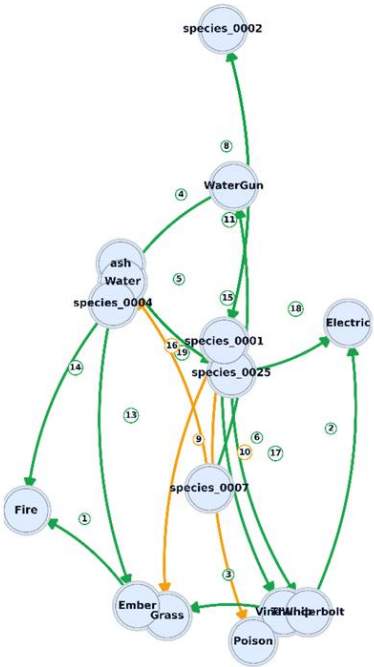
Predicted Graph - story_09 (green=TP, red=FP)



Edge list

- 01. Ember -> Fire : hasMoveType [TP]
- 02. Thunderbolt -> Electric : hasMoveType [TP]
- 03. VineWhip -> Grass : hasMoveType [TP]
- 04. WaterGun -> Water : hasMoveType [TP]
- 05. ash -> species_0025 : ownsPokemon [TP]
- 06. species_0001 -> VineWhip : canLearnMove [TP]
- 07. species_0001 -> species_0002 : evolvesFrom [FP]
- 08. species_0001 -> species_0002 : evolvesTo [TP]
- 11. species_0002 -> species_0001 : evolvesFrom [TP]
- 12. species_0002 -> species_0001 : evolvesTo [FP]
- 13. species_0004 -> Ember : canLearnMove [TP]
- 14. species_0004 -> Fire : hasType [TP]
- 15. species_0007 -> WaterGun : canLearnMove [TP]
- 17. species_0025 -> Thunderbolt : canLearnMove [TP]
- 18. species_0025 -> Electric : hasType [TP]
- 19. species_0025 -> ash : ownedBy [TP]

Actual Graph - story_09 (green=TP, orange=FN)



Edge list

- 01. Ember -> Fire : hasMoveType [TP]
- 02. Thunderbolt -> Electric : hasMoveType [TP]
- 03. VineWhip -> Grass : hasMoveType [TP]
- 04. WaterGun -> Water : hasMoveType [TP]
- 05. ash -> species_0025 : ownsPokemon [TP]
- 06. species_0001 -> VineWhip : canLearnMove [TP]
- 08. species_0001 -> species_0002 : evolvesTo [TP]
- 09. species_0001 -> Grass : hasType [FN]
- 10. species_0001 -> Poison : hasType [FN]
- 11. species_0002 -> species_0001 : evolvesFrom [TP]
- 13. species_0004 -> Ember : canLearnMove [TP]
- 14. species_0004 -> Fire : hasType [TP]
- 15. species_0007 -> WaterGun : canLearnMove [TP]
- 16. species_0007 -> Water : hasType [FN]
- 17. species_0025 -> Thunderbolt : canLearnMove [TP]
- 18. species_0025 -> Electric : hasType [TP]
- 19. species_0025 -> ash : ownedBy [TP]

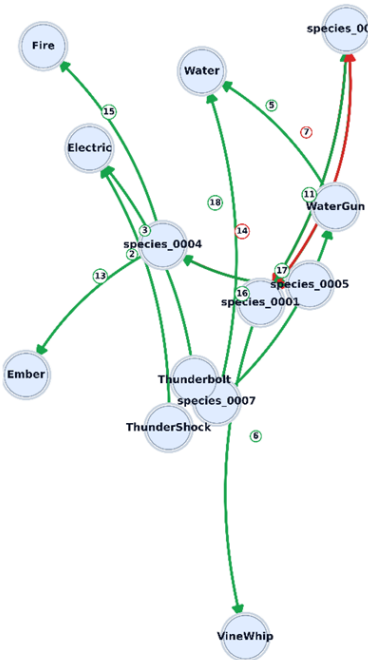
Results Funnel Transformer model

Story	TP	FP	FN	Precision (%)	Recall (%)	F1 (%)
story_01	9	4	0	69.2 %	100 %	81.8 %
story_02	10	7	0	58.8 %	100 %	74.1 %
story_03	11	3	0	78.6 %	100 %	88.0 %
story_04	12	4	2	75.0 %	85.7 %	80.0 %
story_05	11	3	6	78.6 %	64.7 %	71.0 %
story_06	14	2	3	87.5 %	82.4 %	84.8 %
story_07	16	4	2	80.0 %	88.9 %	84.2 %
story_08	13	2	2	86.7 %	86.7 %	86.7 %
story_09	17	4	0	81.0 %	100 %	89.5 %
story_10	22	4	1	84.6 %	95.7 %	89.8 %
TOTAL (mean)	-	-	-	78.5 %	89.4 %	83.6 %

Results Funnel Transformer model: story 5

Relation Graph Comparison - story_05 | Model=FUNNEL-TRANSFORMER | TP=11 FP=3 FN=6 | P=0.786 R=0.647 F1=0.710

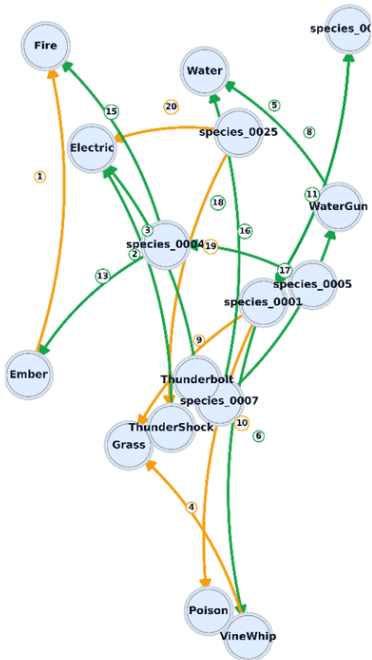
Predicted Graph - story_05 (green=TP, red=FP)



Edge list

- 02. ThunderShock -> Electric : hasMoveType [TP]
- 03. Thunderbolt -> Electric : hasMoveType [TP]
- 05. WaterGun -> Water : hasMoveType [TP]
- 06. species_0001 -> VineWhip : canLearnMove [TP]
- 07. species_0001 -> species_0002 : evolvesFrom [FP]
- 08. species_0001 -> species_0002 : evolvesTo [TP]
- 11. species_0002 -> species_0001 : evolvesFrom [TP]
- 12. species_0002 -> species_0001 : evolvesTo [FP]
- 13. species_0004 -> Ember : canLearnMove [TP]
- 14. species_0004 -> species_0005 : evolvesFrom [FP]
- 15. species_0004 -> Fire : hasType [TP]
- 16. species_0005 -> species_0004 : evolvesFrom [TP]
- 17. species_0007 -> WaterGun : canLearnMove [TP]
- 18. species_0007 -> Water : hasType [TP]

Actual Graph - story_05 (green=TP, orange=FN)



Edge list

- 01. Ember -> Fire : hasMoveType [FN]
- 02. ThunderShock -> Electric : hasMoveType [TP]
- 03. Thunderbolt -> Electric : hasMoveType [TP]
- 04. VineWhip -> Grass : hasMoveType [FN]
- 05. WaterGun -> Water : hasMoveType [TP]
- 06. species_0001 -> VineWhip : canLearnMove [TP]
- 08. species_0001 -> species_0002 : evolvesTo [TP]
- 09. species_0001 -> Grass : hasType [FN]
- 10. species_0001 -> Poison : hasType [FN]
- 11. species_0002 -> species_0001 : evolvesFrom [TP]
- 13. species_0004 -> Ember : canLearnMove [TP]
- 15. species_0004 -> Fire : hasType [TP]
- 16. species_0005 -> species_0004 : evolvesFrom [TP]
- 17. species_0007 -> WaterGun : canLearnMove [TP]
- 18. species_0007 -> Water : hasType [TP]
- 19. species_0025 -> ThunderShock : canLearnMove [FN]
- 20. species_0025 -> Electric : hasType [FN]

Results : Recap

Model	Precision (%)	Recall (%)	F1-score (%)
XLNet	78.1 %	82.8 %	80.4 %
BART	79.2 %	88.1 %	83.4 %
DeROBERTa v3	78.7 %	95.4 %	86.2 %
T5	78.1 %	49.7 %	60.7 %
BigBird	79.7 %	83.4 %	81.6 %
Funnel Transformer	78.5 %	89.4 %	83.6 %

⇒ Precision is quite similar for all models

⇒ Recall is a deciding factor, very good for DeROBERTa v3, Funnel Transformer and BART (as expected from the papers I read)

Table of content

01

**Creation of the
Dataset**

02

SpaCy model

03

**BERT-like
model**

04

Metrics

05

Results

06

Conclusion

Limitations

- Many false-positive patterns across relation type (mention of “Ash” in the text = almost always a wrong relation detected)
- Class imbalance effect, high recall models like DeBERTa v3 may over-predict minority labels

Improvements

- Rebalance training more explicitly, decision when trained made on the macro f1
- Try hybrid baseline with GLiNER (entity extraction) + GLiREL (relation extraction) to compare transformer classifier pipelines vs modular extraction pipelines
- Go deeper into the literature to identify other approaches I may have missed.

Thank you for your attention !

Questions ?

Sources

- [Investigation of improving the pre-training and fine-tuning of BERT model for biomedical relation extraction | BMC Bioinformatics | Springer Nature Link](#)
- [Biomedical Text NER Tagging Tool with Web Interface for Generating BERT-Based Fine-Tuning Dataset](#)
- HF for all the models
- Zotero articles for the choice of all the different transformers